

Implementing a Digital Twin in Node-RED for On-Off Control of an LED Using ESP32

Abstract

This manuscript presents the development of a digital twin system for the on-off control of an LED using Node-RED and ESP32. The project demonstrates the integration of physical hardware, a digital control model, and real-time visualization and control, enabling efficient management of a simple IoT-based system.

1 Introduction

1.1 Digital Twin Overview

A digital twin is a virtual replica of a physical system, providing real-time synchronization between the digital model and the physical entity. This concept is essential for IoT applications as it enables remote monitoring, control, and optimization of systems.

1.2 Objective

The objective of this project is to create a digital twin for a simple on-off LED control system using:

- **ESP32:** A microcontroller for physical control and communication.
- **Node-RED:** A flow-based tool for digital modeling and user interface creation.

2 Hardware Components

2.1 ESP32 Microcontroller

Specifications:

- Dual-core processor.
- Built-in Wi-Fi and Bluetooth for communication.
- Multiple GPIO pins for interfacing.

Use: Acts as the physical interface for LED control and MQTT communication.

2.2 LED

Specifications:

- Standard single-color LED.
- Operates at 3.3V to 5V.

Use: Provides visual feedback for the system.

3 Software Components

3.1 Node-RED

Open-source tool for creating IoT applications. Used to model the digital twin and create a user interface for LED control.

3.2 MQTT Protocol

Lightweight messaging protocol ideal for IoT communication. Used for bidirectional communication between Node-RED and ESP32.

3.3 Arduino IDE

Used to program the ESP32 for LED control and MQTT communication.

4 Implementation Steps

4.1 Hardware Setup

- Connect the positive terminal of the LED to ESP32 GPIO pin 2.
- Connect the negative terminal to the ground via a resistor (220Ω).

4.2 ESP32 Programming

Install the required libraries (WiFi.h for network connection, PubSubClient.h for MQTT communication). Below is the code for ESP32 programming:

```
#include <WiFi.h>
#include <PubSubClient.h>

const char* ssid = "YourSSID";
const char* password = "YourPassword";
const char* mqtt_server = "YourMQTTBrokerIP";

WiFiClient espClient;
PubSubClient client(espClient);

const int ledPin = 2;

void setup() {
    pinMode(ledPin, OUTPUT);
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
    }
    client.setServer(mqtt_server, 1883);
    client.setCallback(callback);
    while (!client.connected()) {
        client.connect("ESP32_LED_Controller");
    }
    client.subscribe("led/control");
}

void callback(char* topic, byte* payload, unsigned int length) {
    String message = "";
    for (int i = 0; i < length; i++) {
        message += (char)payload[i];
    }
    if (message == "ON") {
        digitalWrite(ledPin, HIGH);
    } else if (message == "OFF") {
        digitalWrite(ledPin, LOW);
    }
}
```

```
void loop() {  
    client.loop();  
}
```

4.3 Node-RED Digital Twin Development

4.3.1 Set Up MQTT Nodes

- Add an **MQTT Output** node to publish control commands (ON/OFF) to the topic `led/control`.
- Add an **MQTT Input** node to subscribe to feedback (optional).

4.3.2 Create Control Dashboard

- Use **ui.button** nodes for ON and OFF actions.
- Configure the button to send `ON` or `OFF` messages to the `led/control` topic.

4.3.3 Simulate LED State

- Use a **switch node** to toggle the digital state of the LED in Node-RED.
- Update the state in a **text node** or **LED indicator** on the dashboard.

5 Results

The digital twin effectively controls the physical LED using the Node-RED dashboard. The system provides real-time feedback on the LED state, allowing seamless interaction and monitoring.

6 Conclusion

This project successfully demonstrates the use of Node-RED and ESP32 to implement a digital twin for a simple on-off LED control system. The system can serve as a foundation for more complex IoT applications involving additional sensors and actuators.

7 Future Work

- Extend the system to control multiple LEDs.
- Incorporate cloud integration for remote access.
- Add real-time analytics and logging for LED state changes.